

University of Sri Jayewardenepura – Faculty of Applied Sciences
BSc (General) Degree First Year
CSC 1012 – Introduction to Computer Programming
Project Report: Smart Hospital & Resource Allocation System

1) High -Level Architecture

I have designed the system as a menu-driven console application programmed in C. The system does not have all logic in a single code module but rather split into six files, each of which handles a specific group of functions. The files are related to each other through the common header file, `hospital.h`, which contains constant definitions and external declarations for arrays used in the application plus prototypes of all functions used in the program.

The modular breakdown is as follows

- `main.c` - holds global data arrays (specialty/ward lookup tables, bed occupancy matrix, patient tracking arrays) and the `main()` function, which runs the menu loop and dispatches the choice of the user to relevant module
- `register.c` - handle patient registration, collecting and validating patient details, specialty, selection, and ward admission details.
- `billing.c` - performance all billing and waiting time calculations, and prints the formatted patient bill
- `bed.c` - manages bed allocation (assigning the first available bed in a ward) and displays bed occupancy status per ward.
- `report.c` - sorts registered patients by triage priority and generates the hospital performance summary report
- `fileio.c` - saves and loads bed occupancy status between program runs, and appends every finalized bill to a patient records log
- `hospital.h` - central header file with constants, extern array declarations, and all function prototypes.

The main menu provides five options: Register Patient, Check Bed Status, View Patients by Priority, Generate Performance Report, and Exit. After the bill, bed status, or priority or the report is displayed, the program waits for the user to press Enter to return to the menu, so that they have a chance to read the information presented on the screen.

1.1 List of Implemented Function

Function	File	Purpose
<code>getValidInt()</code>	<code>register.c</code>	Reusable input-validation helper: re-prompts on non-numeric input or values outside a given min/max range.
<code>registerPatients()</code>	<code>register.c</code>	Collects patient name, age, triage level, specialty, and ward/admission details with input validation.

allocatedBed()	bed.c	Find the first available bed in the chosen ward and marks it occupied, return -1 if the ward is full.
checkBedStatus()	bed.c	Displays total, occupied, and available bed counts for each ward.
calculateWaitingTime()	billing.c	Computes estimated waiting time from the current specialty queue count and average consultation time.
calculateEmergencySurcharge()	billing.c	Computes the emergency surcharge based on triage level (0%, 20%, or 50% of base fee).
calculateTotalWardStayCost()	billing.c	Computes ward stay cost as days admitted $\times$ ward daily rate (0 if not admitted).
calculateGrossTotalBill()	billing.c	Sums base fee, surcharge, and ward cost into the gross total bill.
calculateSubsidyDiscount()	billing.c	Applies a 15% discount on the gross total for patients under 5 or over 65 years old.
calculateFinalAmountPayable()	billing.c	Subtracts the discount from the gross total to get the final payable amount.
printBill()	billing.c	prints the formatted admission and billing summary for a patient, matching the required output layout.
sortAndDisplayByPriority()	report.c	Uses selection sort (on an index array, not the original data) to list patients by triage priority, preserving registration order for ties.
generatesSummaryReport()	report.c	Display total patients by triage level, total revenue and discounts, bed occupancy percentage per ward, and the highest highest-paying patient.
savedBedStatus() loadBedStatus()	fileio.c	Persist and restore the bed occupancy from bed_status.txt across program runs.
appendPatientRecord()	fileio.c	Appends each patient's final bill to patient_records.txt as a permanent log.

2)Data Structure Design

The assignment allows using either structs or parallel arrays, this implementation uses the latter throughout, indexed either by patient index i or by ward, specialty id-1 for each. No structs are used.

2.1 Lookup Data

Array	Type & Size	Purpose
specialyNames[4][50]	char [][]	Names of the four doctor specialties (fixed by the assignment table).
specialtyConslationfee[4]	float[]	Base consultation fee per specialty.
specialtyConsationTime[4]	int[]	Average consultation time per patient in minutes.
specialtyPatientCapacity[4]	int[]	Daily patient capacity per specialty.
wardName[4][50]	char [][]	Names of the four hospital wards (fixed by the assignment table).
wardDailyBedRate[4]	float[]	Daily bed rate per ward.
wardTotalBedCapacity[4]	int[]	Total bed capacity per ward (20,10,10,5)

2.2 Bed Occupancy Matrix

bedOccupancy[4][20] is a 2D integer array. Each row represents a ward, and each column represents a slot (0 = available, 1 = occupied). The size of the column is 20, as the biggest capacity among all the wards is 20 according to the assignment. Therefore, the number of columns for smaller wards will be less than 20, but it will be bounded by the wardTotalBedCapacity[] in every loop when accessing this array.

2.3 Patient Tracking

Array	Type & Size	Purpose
patientName[100][50]	char [][]	Patient names, indexed by registration order.
patientAge[100]	int[]	Patient age.
patientEmergencyLevel[100]	int[]	Triage level (1 Normal, 2 Urgent, 3 Critical).
specialtyId[100]	int[]	Chosen specialty Id (1-4)
isAdmitted[100]	int[]	1 if admitted to a ward, 0 if outpatient.

wardId[100]	int[]	Chosen Ward Id (0 if not admitted).
daysAdmitted[100]	int[]	Number of days admitted (0 if not admitted)
patientFinalBill[100]	float[]	Final payable amount, stored for the summary report.
patientDiscount[100]	float[]	Age subsidy discount granted, stored for summary report.
specialtyQueueCount[4]	int[]	Running count of patients queued per specialty, incremented after each registration.

Each of the patient array has the same index i , and therefore it is always possible to reconstruct an entire patient's records by using the `patientName[i]`, `patientAge[i]`, etc. Additionally, a single `patientCount` variable keeps track of how many of the `MAX_PATIENTS` are in use.

3. Github Repository

Repository URL: <https://github.com/RansikaTD-23/Smart-Hospital-Resource-Allocation-System>

4. Assumptions Made During Development

- Assign `MAX_PATIENTS` to 100 because it is a reasonable maximum number of patients for one cycle of patient processing. This value was not specified in the assignment and was selected as a convenient default.
- `NAME_LENGTH` is set to 50 characters, assumed sufficient for realistic patient, specialty, and ward names.
- Bed numbers are assigned as the first available (lowest-numbered) free bed in the selected ward, since the assignment does not specify an allocation strategy.
- If a ward is full at the time of admission, the system displays a message and does not allocate a bed handling this case beyond notifying the user was not specified in the assignment.
- Patient age is validated to fall between 0 and 120 years, and days admitted must be a positive integer, as reasonable data-entry safeguards beyond the assignment's explicit requirements.
- Patient IDs are generated automatically as PAT-1001, PAT-1002, and so on, based on registration order, matching the sample output format.
- The bonus file-handling requirement was implemented, bed occupancy is saved to `beds_status.txt` on exit and reloaded on startup, and every finalized bill is appended to `patient_records.txt`.
- Persistence of files is limited as mentioned above, meaning that only bed occupancy and billing history can be stored across different executions. Patient information and the list of priorities and performance report will be calculated from the memory arrays at every execution and not stored, because it is not needed for the task.

- The information regarding how to address the patient (Mr, Mrs, or Miss) is collected separately from the name of the patient and joined together during the storage and displaying of the patient information to meet the format of the sample bill.
- The validation logic for all the numerical prompts is done with the help of a common validation technology that prompts again in case of either a value being out of the range or user entering non-numeric digits thus preventing the occurrence of non-numeric characters in the input process.
- Instead of combining priority sorting and performance reporting options into a single option, the main menu has been split into five different options so that different reports can be viewed and assessed separately.